

# Software Design Specification

## Analytics & Community Insights

Sub-Project Module Design

Verified neighborhood community platform

|             |                                   |
|-------------|-----------------------------------|
| Document ID | SP-SDS-ACI-v1.0                   |
| Version     | 1.0 (Final Submission)            |
| Date        | March 2026                        |
| Standard    | IEEE Std 1016-2009                |
| Team Name   | <b>KODERS</b>                     |
| Module      | Analytics & Community<br>Insights |



## Team Members

| Name        | Roll No. | Role                     |
|-------------|----------|--------------------------|
| Shah Faisal | 23I-0058 | Team Lead                |
| Hamad Khan  | 23I-3095 | Analytics Backend Dev    |
| Ali Sajjad  | 23I-0110 | Data Pipeline Engineer   |
| Shace Tariq | 23I-3069 | Frontend / Dashboard Dev |

Prepared for the **Software Project Management Course Project**

National University of Computer and Emerging Sciences (NUCES) — Section A

This document constitutes the detailed design specification for the Analytics & Community Insights sub-project of the Verified neighborhood community platform platform, prepared in accordance with IEEE Std 1016-2009.

# Contents

|          |                                         |           |
|----------|-----------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                     | <b>2</b>  |
| 1.1      | Purpose . . . . .                       | 2         |
| 1.2      | Scope . . . . .                         | 2         |
| 1.3      | Overview . . . . .                      | 3         |
| 1.4      | Reference Material . . . . .            | 4         |
| 1.5      | Definitions and Acronyms . . . . .      | 4         |
| <b>2</b> | <b>System Overview</b>                  | <b>7</b>  |
| <b>3</b> | <b>System Architecture</b>              | <b>9</b>  |
| 3.1      | Architectural Design . . . . .          | 9         |
| 3.2      | Decomposition Description . . . . .     | 9         |
| 3.3      | Design Rationale . . . . .              | 11        |
| <b>4</b> | <b>Data Design</b>                      | <b>12</b> |
| 4.1      | Data Description . . . . .              | 12        |
| 4.2      | Data Dictionary . . . . .               | 13        |
| <b>5</b> | <b>Component Design</b>                 | <b>18</b> |
| 5.1      | Component 1: Event Consumer . . . . .   | 18        |
| 5.2      | Component 2: Data Aggregator . . . . .  | 19        |
| 5.3      | Component 3: Trend Detector . . . . .   | 21        |
| 5.4      | Component 4: Report Generator . . . . . | 23        |
| 5.5      | Component 5: Analytics API . . . . .    | 24        |
| <b>6</b> | <b>Human Interface Design</b>           | <b>27</b> |
| 6.1      | Overview of User Interface . . . . .    | 27        |
| 6.2      | Screen Images . . . . .                 | 27        |
| 6.3      | Screen Objects and Actions . . . . .    | 29        |
| <b>7</b> | <b>Requirements Matrix</b>              | <b>31</b> |
| <b>8</b> | <b>Appendices</b>                       | <b>34</b> |

# 1 Introduction

## 1.1 Purpose

This **Software Design Specification (SP-SDS)** document describes the architectural and detailed design of the **Analytics & Community Insights** module for the *Verified Neighborhood Community Platform*. The document is prepared in accordance with **IEEE Std 1016-2009** and serves as the authoritative technical reference for implementation, review, and integration activities.

### Intended Audience:

- The **KODERS** development team responsible for implementing this module.
- The **Integration Group** responsible for merging all sub-project designs into a unified system.
- **Instructors and academic evaluators** reviewing the sub-project for Assignment 2.
- **Future maintainers** or contributors to the platform.

This document provides sufficient design detail to enable developers to implement the Analytics & Community Insights module without requiring further design decisions, and to enable integration engineers to connect this module with peer modules via defined API contracts.

## 1.2 Scope

The **Analytics & Community Insights module** (hereafter “ACI module”) is a backend-and-dashboard sub-system of the platform. Its scope encompasses:

1. **Data Collection & Ingestion:** Aggregating event-driven and batch data from all modules (User Management, Post/Feed, Incident Reporting, Marketplace, Messaging, and Moderation).
2. **Data Processing & Storage:** Transforming raw operational data into aggregated time-series metrics and persisting them in an analytics data store.
3. **Trend Detection & Anomaly Analysis:** Computing statistical trends in

community engagement, safety incidents, and marketplace activity; raising alerts on anomalous patterns.

4. **Insight Generation:** Producing pre-computed and on-demand analytical reports covering community health, engagement rates, incident frequency, and moderation effectiveness.
5. **Admin & Moderator Dashboards:** Exposing analytics data through interactive, role-filtered web dashboards accessible to neighborhood administrators and platform moderators.
6. **RESTful Analytics API:** Providing versioned API endpoints consumable by other modules (e.g., the AI Moderation module) and the mobile/web frontend.

**⚠ Out of Scope:** The ACI module does *not* handle raw user authentication, incident lifecycle management, or content moderation decisions — it *consumes* data produced by those modules.

### 1.3 Overview

This document is organized as follows:

#### §1 — System Overview

High-level description of the platform and the role of the ACI module within it.

#### §2 — System Architecture

Architectural design, module decomposition, and design rationale.

#### §3 — Data Design

Entity descriptions, data models, and a complete data dictionary.

#### §4 — Component Design

Pseudocode and algorithmic descriptions of all major ACI components.

#### §5 — Human Interface Design

Dashboard layout, screen descriptions, and UI behavior specifications.

#### §6 — Requirements Matrix

Traceability from functional requirements to design components.

## §7 — Appendices

Supplementary material including API contracts and error codes.

### 1.4 Reference Material

1. *Verified Neighborhood Community Hub, Business Case*, Muhammad Hamza Khan (Roll: 23I-3032), Section A.
2. *Assignment 2 Specification*, Software Engineering Course, NUCES, March 2026.
3. IEEE Std 1016-2009, *IEEE Standard for Information Technology – Systems Design – Software Design Descriptions*.
4. IEEE Std 830-1998, *Recommended Practice for Software Requirements Specifications*.
5. Martin Fowler, *Patterns of Enterprise Application Architecture*, Addison-Wesley, 2002.
6. Sam Newman, *Building Microservices*, O'Reilly Media, 2015.
7. Apache Kafka Documentation, v3.6. <https://kafka.apache.org/documentation/>
8. Redis Documentation, v7.2. <https://redis.io/docs/>
9. PostgreSQL 16 Documentation. <https://www.postgresql.org/docs/16/>

### 1.5 Definitions and Acronyms

| Term / Acronym | Definition                                                                                                 |
|----------------|------------------------------------------------------------------------------------------------------------|
| ACI            | Analytics & Community Insights — the module described in this document.                                    |
| API            | Application Programming Interface — a defined interface through which software components communicate.     |
| DAU            | Daily Active Users — count of unique verified users who performed at least one action in a 24-hour window. |

*Continued on next page...*

| Term / Acronym     | Definition                                                                                                 |
|--------------------|------------------------------------------------------------------------------------------------------------|
| <b>MAU</b>         | Monthly Active Users — same metric over a 30-day rolling window.                                           |
| <b>ER</b>          | Engagement Rate — ratio of interactions (likes, comments, shares) to impressions for a post or feed.       |
| <b>ETL</b>         | Extract, Transform, Load — the pipeline that moves data from operational systems into the analytics store. |
| <b>JWT</b>         | JSON Web Token — compact, URL-safe means of representing claims for authorization.                         |
| <b>KPI</b>         | Key Performance Indicator — a measurable value that demonstrates effectiveness.                            |
| <b>OLAP</b>        | Online Analytical Processing — query patterns optimized for multi-dimensional aggregation.                 |
| <b>OLTP</b>        | Online Transaction Processing — query patterns optimized for high-frequency transactional reads/writes.    |
| <b>REST</b>        | Representational State Transfer — an architectural style for distributed hypermedia systems.               |
| <b>SDD</b>         | Software Design Description — synonymous with SDS in this document context.                                |
| <b>SDS</b>         | Software Design Specification — this document.                                                             |
| <b>SP-SDS</b>      | Sub-Project Software Design Specification — a module-level SDS (this document).                            |
| <b>SP-SRS</b>      | Sub-Project Software Requirements Specification — the companion requirements document.                     |
| <b>RBAC</b>        | Role-Based Access Control — permissions assigned via user roles (resident, moderator, admin).              |
| <b>Time-series</b> | A sequence of data points indexed by timestamp, used for trend analysis.                                   |

*Continued on next page...*

| Term / Acronym |  | Definition                                                                                |
|----------------|--|-------------------------------------------------------------------------------------------|
| Anomaly        |  | A data point or sequence that deviates significantly from expected behavior.              |
| Z-score        |  | A statistical measure expressing how many standard deviations a value is from the mean.   |
| EMA            |  | Exponential Moving Average — a weighted moving average giving more weight to recent data. |
| Neighborhood   |  | A geographically defined community unit within , verified by address.                     |

## 2 System Overview

---

### The Neighbourhood Community Platform

---

Verified neighborhood community platform designed to replace fragmented WhatsApp groups with a secure, moderated, and feature-rich digital neighborhood hub. Pakistani urban communities currently rely on informal messaging groups that lack user verification, content moderation, structured history, and safety coordination. Verified neighborhood community platform addresses these gaps by providing:

- **Address-verified** user registration ensuring all members belong to the physical neighborhood.
- A **neighborhood-scoped post/feed system** for local news, announcements, and discussions.
- An **incident and safety reporting system** with sub-10-second alert delivery.
- A **classified marketplace** for buying, selling, and exchanging goods among verified neighbors.
- A **local services directory** for discovering neighborhood businesses and service providers.
- An **AI-powered moderation layer** for spam, scam, and toxicity detection.
- An **analytics and insights layer** (this module) providing data-driven visibility into community health.

### Role of the ACI Module

---

The Analytics & Community Insights module occupies the **observability and intelligence tier** of the Verified neighborhood community platform architecture. It is a read-heavy consumer of data produced by all other modules. Its primary consumers are neighborhood administrators and platform moderators who require actionable, data-backed insights to govern their communities effectively.

#### ACI Module – Key Capabilities

- **Administrators** can monitor neighborhood engagement KPIs, identify low-activity periods, and plan community initiatives.
- **Moderators** can detect surges in incident reporting, identify recurring offenders, and assess the impact of moderation actions.
- **The platform itself** can supply the AI Moderation and Smart Feed Ranking modules with pre-computed community behavior signals.

**Important:** The ACI module has *no authority* to modify any operational data. It is strictly a consumer and reporter. All writes to operational systems (user records, posts, incidents) remain the responsibility of their respective modules.

## 3 System Architecture

### 3.1 Architectural Design

The Verified neighborhood community platform platform employs a **modular microservices architecture** with clearly bounded service contexts. Each module exposes REST APIs and publishes domain events to a shared message broker. The ACI module subscribes to these events to build its analytical data store.

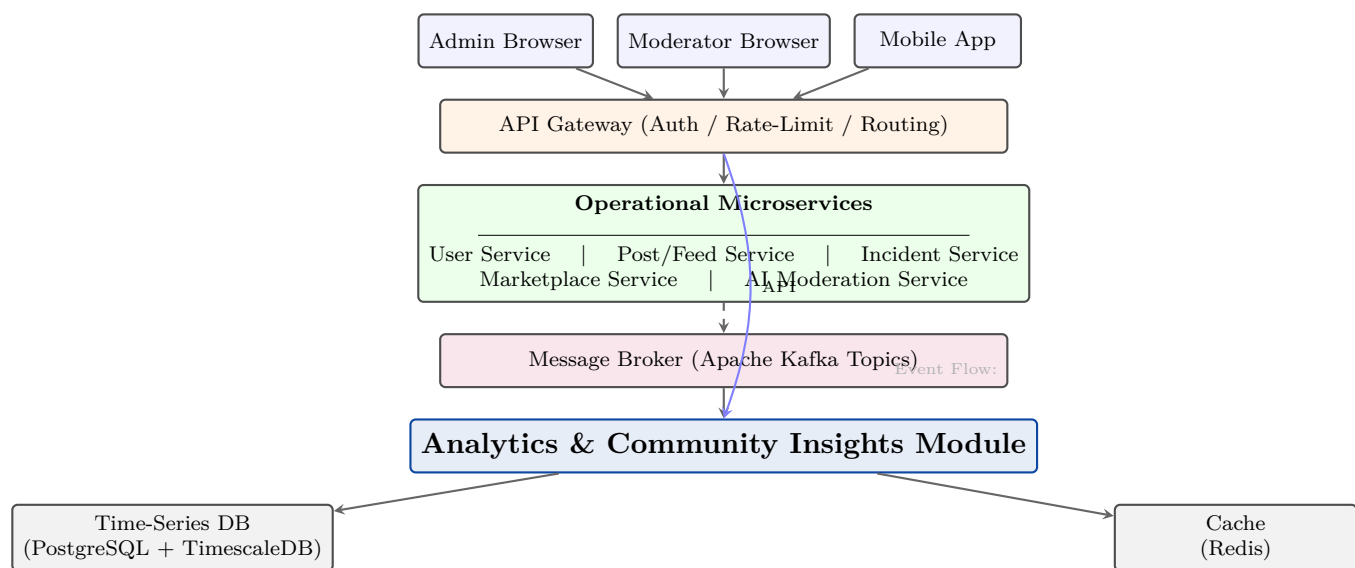


Figure 1: High-Level System Architecture – Platform (ACI module highlighted)

#### Key Architectural Properties:

- **Event-driven ingestion:** Operational modules publish domain events to Kafka topics. ACI subscribes to relevant topics, ensuring loose coupling.
- **CQRS separation:** ACI maintains its own read-optimized data store, decoupled from operational OLTP databases.
- **API Gateway integration:** ACI exposes REST endpoints behind the shared API Gateway for authentication and rate limiting.
- **Caching layer:** Frequently requested dashboard metrics cached in Redis for sub-200ms load times.

### 3.2 Decomposition Description

The ACI module is internally decomposed into **five cooperating sub-components**:

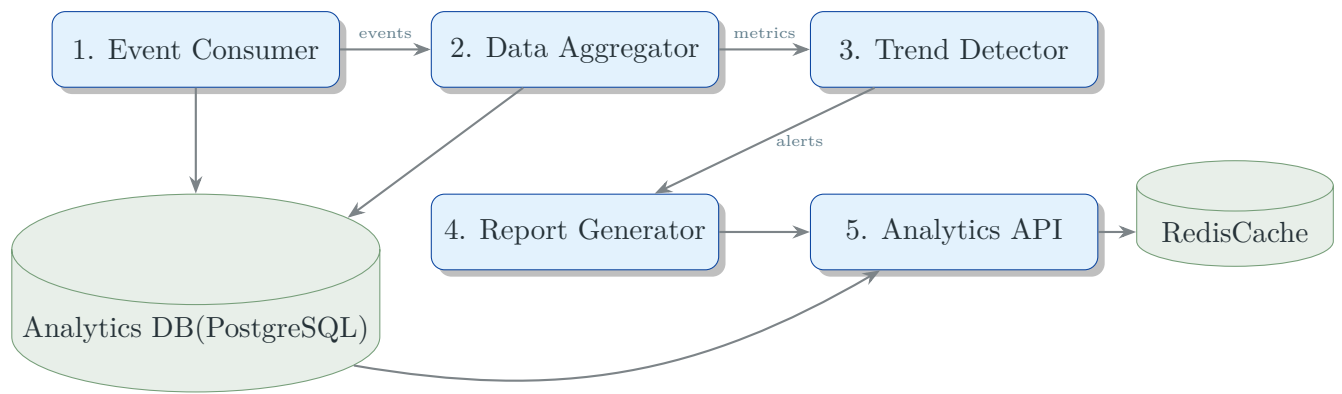


Figure 2: ACI Module – Internal Component Decomposition

### 1. Event Consumer

A Kafka consumer group that subscribes to all Verified neighborhood community platform domain event topics. It deserializes incoming events, validates their schema, and routes them to the Data Aggregator or writes raw events to the analytics database.

### 2. Data Aggregator

A scheduled processing engine (running on 1-minute, 5-minute, hourly, and daily cadences) that reads raw event records and computes aggregated metrics: DAU, MAU, posts per neighborhood per hour, incident counts by category, marketplace listing activity, and moderation action throughput.

### 3. Trend Detector

A statistical analysis component that applies Z-score analysis and Exponential Moving Average (EMA) computation to time-series metric data. It identifies upward/downward engagement trends and raises anomaly alerts when any metric deviates by more than  $3\sigma$  from its historical baseline.

### 4. Report Generator

Produces pre-formatted analytical reports on scheduled intervals (daily, weekly, monthly) and on-demand. Reports are structured JSON objects stored in the database and optionally exported as PDF summaries.

### 5. Analytics API

A versioned RESTful API layer (Express.js / Node.js) that exposes metric queries, report retrieval, and anomaly alert endpoints to the API Gateway and dashboard frontend.

### 3.3 Design Rationale

| Design Decision                              | Rationale                                                                                                                                                                                                         |
|----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Event-driven ingestion via Kafka</b>      | Decouples ACI from operational services. Services publish events and forget; ACI subscribes independently without adding latency to transactional flows. Supports replay for backfill and fault tolerance.        |
| <b>Separate analytics data store</b>         | Prevents complex OLAP queries from competing with OLTP workloads on operational databases. Enables schema optimized for time-series aggregation (e.g., partitioned by neighborhood and day).                      |
| <b>PostgreSQL with TimescaleDB extension</b> | Provides native time-series partitioning and continuous aggregate views while retaining full SQL expressiveness. Free-tier compatible with university infrastructure.                                             |
| <b>Redis caching</b>                         | Dashboard KPIs queried at high frequency must return in < 200 ms. Redis TTL-based caching (60-second TTL for real-time metrics, 5-minute TTL for daily summaries) satisfies this without repeated DB aggregation. |
| <b>Z-score anomaly detection</b>             | Statistically principled, parameter-light, and interpretable. Suitable for the volume of data expected in a neighborhood-scale deployment (hundreds to thousands of events per day per neighborhood).             |
| <b>RESTful API (not GraphQL)</b>             | Team familiarity, simpler caching semantics, and straightforward integration with peer modules that already expose REST endpoints.                                                                                |
| <b>Modular ETL cadences</b>                  | Different metrics have different freshness requirements. Real-time event counts (1-minute cadence), engagement summaries (hourly), and KPI reports (daily) balance accuracy with processing cost.                 |

## 4 Data Design

### 4.1 Data Description

The ACI module maintains its own **analytics schema**, logically separated from the operational database schemas of peer modules. Data flows into the ACI schema through the ETL pipeline and is never written back to any operational schema.

#### *Primary Entity Relationships*

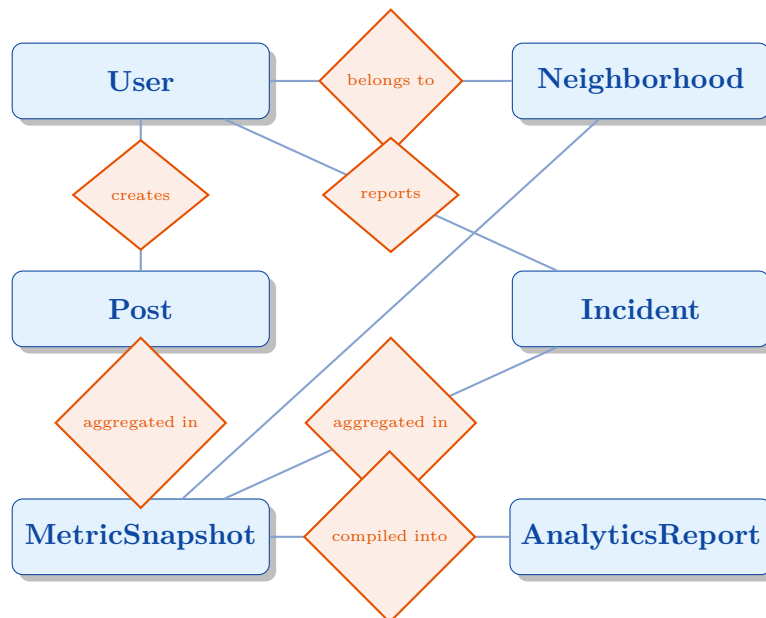


Figure 3: ACI Data Model — Entity Relationships

#### *Entity Descriptions*

##### User (shadow copy)

A read-only projection of verified user data sourced from the User Management module via Kafka events. Stores only the fields required for analytics segmentation (user ID, neighborhood ID, registration date, verification status).

##### Neighborhood

A geographic community unit identified by ID and name. All analytics metrics are scoped to a neighborhood.

## Post

An event-sourced record representing a post lifecycle event (created, liked, commented, shared, deleted, flagged). Timestamp-indexed for time-series aggregation.

## Incident

An event-sourced record representing incident report events (submitted, categorized, resolved, escalated). Includes category, severity, and resolution time.

## MetricSnapshot

A pre-computed, time-windowed aggregation record. Each row represents one metric (e.g., `dau`, `post_count`, `incident_count`) for one neighborhood over one time window (1h, 24h, 7d, 30d).

## AnalyticsReport

A structured JSON document representing a full analytical report for a neighborhood and time period. Includes KPI summaries, trend indicators, anomaly flags, and top-contributor rankings.

## AnomalyAlert

A record generated by the Trend Detector when a metric deviation exceeds the configured threshold. Consumed by the dashboard and optionally forwarded to the Notification Service.

## 4.2 Data Dictionary

| Field Name                                                                                 | Type      | Nullable | Description                                     |
|--------------------------------------------------------------------------------------------|-----------|----------|-------------------------------------------------|
| <div> <div></div> <div>Table: <code>aci_users</code> (Shadow User Projection)</div> </div> |           |          |                                                 |
| <code>user_id</code>                                                                       | UUID      | NO       | Unique identifier mirrored from User Service.   |
| <code>neighborhood_id</code>                                                               | UUID      | NO       | Foreign key to <code>aci_neighborhoods</code> . |
| <code>registered_at</code>                                                                 | TIMESTAMP | TZ NO    | Timestamp of verified registration.             |

*Continued...*

| Field Name                                                                                                        | Type         | Nullable | Description                                                                                                                                    |
|-------------------------------------------------------------------------------------------------------------------|--------------|----------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>is_verified</code>                                                                                          | BOOLEAN      | NO       | Address verification status.                                                                                                                   |
| <code>is_active</code>                                                                                            | BOOLEAN      | NO       | Whether the user has been deactivated.                                                                                                         |
|  <b>Table: aci_neighborhoods</b> |              |          |                                                                                                                                                |
| <code>neighborhood_id</code>                                                                                      | UUID         | NO       | Primary key. Mirrors the Neighborhood Service ID.                                                                                              |
| <code>name</code>                                                                                                 | VARCHAR(120) | NO       | Human-readable neighborhood name (e.g., “DHA Phase 5, Lahore”).                                                                                |
| <code>city</code>                                                                                                 | VARCHAR(8)   | NO       | City the neighborhood belongs to.                                                                                                              |
| <code>created_at</code>                                                                                           | TIMESTAMPTZ  | NO       | Timestamp when the neighborhood was first activated.                                                                                           |
|  <b>Table: aci_post_events</b> |              |          |                                                                                                                                                |
| <code>event_id</code>                                                                                             | UUID         | NO       | Unique event identifier from the event stream.                                                                                                 |
| <code>post_id</code>                                                                                              | UUID         | NO       | ID of the post this event pertains to.                                                                                                         |
| <code>neighborhood_id</code>                                                                                      | UUID         | NO       | Scoped neighborhood.                                                                                                                           |
| <code>author_id</code>                                                                                            | UUID         | YES      | User ID of post author (null if user deleted).                                                                                                 |
| <code>event_type</code>                                                                                           | VARCHAR(3)   | NO       | Enum: <code>created</code> , <code>liked</code> , <code>commented</code> , <code>shared</code> , <code>flagged</code> , <code>deleted</code> . |
| <code>occurred_at</code>                                                                                          | TIMESTAMPTZ  | NO       | Timestamp of the event (from event source).                                                                                                    |

*Continued...*

| Field Name                                                                                                             | Type        | Nullable | Description                                                                                                                         |
|------------------------------------------------------------------------------------------------------------------------|-------------|----------|-------------------------------------------------------------------------------------------------------------------------------------|
| <code>ingested_at</code>                                                                                               | TIMESTAMPTZ | NO       | Timestamp when the ACI module ingested the event.                                                                                   |
|  <b>Table: aci_incident_events</b>    |             |          |                                                                                                                                     |
| <code>event_id</code>                                                                                                  | UUID        | NO       | Unique event identifier.                                                                                                            |
| <code>incident_id</code>                                                                                               | UUID        | NO       | ID of the reported incident.                                                                                                        |
| <code>neighborhood_id</code>                                                                                           | UUID        | NO       | Scoped neighborhood.                                                                                                                |
| <code>reporter_id</code>                                                                                               | UUID        | YES      | Reporting user ID.                                                                                                                  |
| <code>category</code>                                                                                                  | VARCHAR(50) | NO       | Incident category: <code>theft</code> , <code>noise</code> , <code>emergency</code> , <code>vandalism</code> , <code>other</code> . |
| <code>severity</code>                                                                                                  | SMALLINT    | NO       | Severity level: 1 (low) to 5 (critical).                                                                                            |
| <code>event_type</code>                                                                                                | VARCHAR(30) | NO       | Enum: <code>submitted</code> , <code>categorized</code> , <code>resolved</code> , <code>escalated</code> .                          |
| <code>occurred_at</code>                                                                                               | TIMESTAMPTZ | NO       | Timestamp of the event.                                                                                                             |
|  <b>Table: aci_metric_snapshots</b> |             |          |                                                                                                                                     |
| <code>snapshot_id</code>                                                                                               | BIGSERIAL   | NO       | Auto-incrementing primary key.                                                                                                      |
| <code>neighborhood_id</code>                                                                                           | UUID        | NO       | Scoped neighborhood.                                                                                                                |
| <code>metric_name</code>                                                                                               | VARCHAR(60) | NO       | Metric identifier, e.g., <code>dau</code> , <code>post_count</code> , <code>incident_count</code> , <code>engagement_rate</code> .  |

*Continued...*

| Field Name                                                                                                            | Type            | Nullable | Description                                                           |
|-----------------------------------------------------------------------------------------------------------------------|-----------------|----------|-----------------------------------------------------------------------|
| <code>window</code>                                                                                                   | VARCHAR(1       | NO       | Aggregation window: 1h, 24h, 7d, 30d.                                 |
| <code>value</code>                                                                                                    | DOUBLE<br>PREC. | NO       | Computed metric value.                                                |
| <code>snapshot_at</code>                                                                                              | TIMESTAM        | NO       | Timestamp of snapshot (window end time).                              |
| <code>computed_at</code>                                                                                              | TIMESTAMPTZ     | NO       | Timestamp when the aggregation job ran.                               |
|  <b>Table: aci_analytics_reports</b> |                 |          |                                                                       |
| <code>report_id</code>                                                                                                | UUID            | NO       | Primary key.                                                          |
| <code>neighborhood_id</code>                                                                                          | UUID            | NO       | Target neighborhood.                                                  |
| <code>period_start</code>                                                                                             | TIMESTAM        | NO       | Report period start.                                                  |
| <code>period_end</code>                                                                                               | TIMESTAMPTZ     | NO       | Report period end.                                                    |
| <code>report_type</code>                                                                                              | VARCHAR(2       | NO       | <b>daily</b> , <b>weekly</b> , or <b>monthly</b> .                    |
| <code>payload</code>                                                                                                  | JSONB           | NO       | Full report data as structured JSON (KPIs, trends, top contributors). |
| <code>generated_at</code>                                                                                             | TIMESTAM        | NO       | Timestamp of report generation.                                       |
| <code>generated_by</code>                                                                                             | VARCHAR(30)     | NO       | <b>scheduler</b> or <b>on_demand</b> .                                |
|  <b>Table: aci_anomaly_alerts</b>  |                 |          |                                                                       |

*Continued...*

| Field Name                   | Type            | Nullable | Description                                                          |
|------------------------------|-----------------|----------|----------------------------------------------------------------------|
| <code>alert_id</code>        | UUID            | NO       | Primary key.                                                         |
| <code>neighborhood_id</code> | UUID            | NO       | Affected neighborhood.                                               |
| <code>metric_name</code>     | VARCHAR(6)      | NO       | Metric that triggered the alert.                                     |
| <code>observed_value</code>  | DOUBLE<br>PREC. | NO       | Metric value at detection time.                                      |
| <code>expected_value</code>  | DOUBLE<br>PREC. | NO       | Baseline expected value ( $\mu$ ).                                   |
| <code>z_score</code>         | DOUBLE<br>PREC. | NO       | Computed Z-score of the deviation.                                   |
| <code>severity</code>        | VARCHAR(1)      | NO       | <b>warning</b> ( $ z  \geq 2$ ) or <b>critical</b> ( $ z  \geq 3$ ). |
| <code>detected_at</code>     | TIMESTAMPTZ     | NO       | Timestamp of detection.                                              |
| <code>acknowledged</code>    | BOOLEAN         | NO       | Whether an admin has acknowledged the alert.                         |
| <code>resolved_at</code>     | TIMESTAMPTZ     | YES      | Null until resolved.                                                 |

## 5 Component Design

### 5.1 Component 1: Event Consumer



#### Component 1 – Event Consumer

**Responsibility:** Subscribe to Kafka topics for all Verified neighborhood community platform domain events, validate event schemas, and persist raw event records to the ACI database.

#### Kafka Topics Consumed:

- `Verified neighborhood community platform.users.events` — user registered, verified, deactivated
- `Verified neighborhood community platform.posts.events` — post created, liked, commented, shared, flagged, deleted
- `Verified neighborhood community platform.incidents.events` — incident submitted, categorized, resolved, escalated
- `Verified neighborhood community platform.marketplace.events` — listing created, viewed, sold
- `Verified neighborhood community platform.moderation.events` — content flagged, removed, user warned/banned

```

1 FUNCTION start_event_consumer():
2     consumer = Kafka.Consumer(
3         group_id = "aci-consumer-group",
4         topics   = [ Verified neighborhood community
5                     platform_USERS_TOPIC, Verified neighborhood community
6                     platform_POSTS_TOPIC,
7                     Verified neighborhood community
8                     platform_INCIDENTS_TOPIC, Verified
9                     neighborhood community
10                    platform_MARKETPLACE_TOPIC,
11                    Verified neighborhood community
12                    platform_MODERATION_TOPIC],
13     auto_offset_reset = "earliest"
14 )

```

```

9
10 WHILE true:
11     batch = consumer.poll(timeout_ms = 500, max_records = 200)
12     FOR event IN batch:
13         TRY:
14             validated = validate_schema(event)    // check
15             required fields
16             route_event(validated)                // dispatch by
17             event_type
18             consumer.commit(event.offset)
19         CATCH SchemaValidationError AS e:
20             log_error("Schema invalid", event, e)
21             send_to_dead_letter_queue(event)
22         CATCH DatabaseError AS e:
23             log_error("DB write failed", event, e)
24             // do NOT commit -- Kafka will redeliver
25
26 FUNCTION route_event(event):
27     SWITCH event.topic:
28         CASE Verified neighborhood community platform_USERS_TOPIC:
29             upsert_aci_users(event)
30         CASE Verified neighborhood community platform_POSTS_TOPIC:
31             insert_post_event(event)
32         CASE Verified neighborhood community
33             platform_INCIDENTS_TOPIC: insert_incident_event(event)
34         CASE Verified neighborhood community
35             platform_MARKETPLACE_TOPIC: insert_marketplace_event(
36                 event)
37         CASE Verified neighborhood community
38             platform_MODERATION_TOPIC: insert_moderation_event(
39                 event)
40         DEFAULT: log_warning("Unhandled topic", event.topic)

```

Listing 1: Event Consumer — Pseudocode

## 5.2 Component 2: Data Aggregator



### Component 2 – Data Aggregator

**Responsibility:** Execute scheduled ETL jobs to compute aggregated metrics from raw event tables and populate `aci_metric_snapshots`.

#### Job Schedule:

- Every **1 minute**: real-time event count per neighborhood.

- Every **hour**: DAU, post count, incident count, engagement rate.
- Every **24 hours**: MAU, top contributors, category distributions.

```

1 FUNCTION run_hourly_aggregation():
2     neighborhoods = db.query("SELECT neighborhood_id FROM
3         aci_neighborhoods")
4     window_end     = now()
5     window_start   = window_end - INTERVAL("24h")
6
7     FOR nbhd IN neighborhoods:
8         metrics = {}
9
10        // Daily Active Users
11        metrics["dau"] = db.query_scalar("""
12            SELECT COUNT(DISTINCT author_id)
13            FROM aci_post_events
14            WHERE neighborhood_id = :nbhd
15            AND occurred_at BETWEEN :start AND :end
16            """, nbhd=nbhd.id, start=window_start, end=window_end)
17
18        // Post Count
19        metrics["post_count"] = db.query_scalar("""
20            SELECT COUNT(*) FROM aci_post_events
21            WHERE neighborhood_id = :nbhd
22            AND event_type = 'created'
23            AND occurred_at BETWEEN :start AND :end
24            """, nbhd=nbhd.id, start=window_start, end=window_end)
25
26        // Incident Count
27        metrics["incident_count"] = db.query_scalar("""
28            SELECT COUNT(*) FROM aci_incident_events
29            WHERE neighborhood_id = :nbhd
30            AND event_type = 'submitted'
31            AND occurred_at BETWEEN :start AND :end
32            """, nbhd=nbhd.id, start=window_start, end=window_end)
33
34        // Engagement Rate = (likes + comments + shares) /
35        post_count
36        interactions = db.query_scalar("""
37            SELECT COUNT(*) FROM aci_post_events
38            WHERE neighborhood_id = :nbhd
39            AND event_type IN ('liked','commented','shared')
40            AND occurred_at BETWEEN :start AND :end
41            """, nbhd=nbhd.id, start=window_start, end=window_end)
42
43        metrics["engagement_rate"] = interactions / MAX(metrics["

```

```

    post_count"], 1)
42
43 // Write snapshots
44 FOR metric_name, value IN metrics.items():
45     db.upsert("aci_metric_snapshots", {
46         neighborhood_id : nbhd.id,
47         metric_name      : metric_name,
48         window           : "24h",
49         value            : value,
50         snapshot_at      : window_end,
51         computed_at      : now()
52     })
53
54 // Invalidate Redis cache for this neighborhood
55 redis.delete("metrics:" + nbhd.id + ":24h")

```

Listing 2: Data Aggregator — Metric Computation Pseudocode

### 5.3 Component 3: Trend Detector

#### Component 3 – Trend Detector

**Responsibility:** Analyse metric time-series to identify statistically significant deviations and generate `aci_anomaly_alerts`.

**Algorithm:** For each metric and neighborhood, maintain a rolling 30-day baseline ( $\mu, \sigma$ ). Compute the Z-score of the latest snapshot. If  $|z| \geq 2$ , raise a *warning*; if  $|z| \geq 3$ , raise a *critical* alert.

```

1 FUNCTION run_trend_detection():
2     FOR each (neighborhood_id, metric_name) pair:
3         // Fetch last 30 days of daily snapshots
4         history = db.query("""
5             SELECT value FROM aci_metric_snapshots
6             WHERE neighborhood_id = :nbhd
7                 AND metric_name      = :metric
8                 AND window           = '24h'
9                 AND snapshot_at      >= now() - INTERVAL('30 days')
10            ORDER BY snapshot_at ASC
11        """)
12
13     IF len(history) < 7:
14         CONTINUE // Insufficient history for reliable baseline
15
16     values = [row.value FOR row IN history]

```

```

17     mu      = mean(values)
18     sigma   = std_dev(values)
19
20     IF sigma == 0:
21         CONTINUE // No variance; skip
22
23     latest  = values[-1]
24     z_score = (latest - mu) / sigma
25
26     IF abs(z_score) >= 3:    severity = "critical"
27     ELIF abs(z_score) >= 2: severity = "warning"
28     ELSE:                  CONTINUE // Within normal bounds
29
30     // Check if an open alert already exists (avoid duplicates)
31     existing = db.query("""
32         SELECT alert_id FROM aci_anomaly_alerts
33         WHERE neighborhood_id = :nbhd
34             AND metric_name      = :metric
35             AND resolved_at IS NULL
36             AND detected_at      >= now() - INTERVAL('2 hours')
37     """)
38
39     IF existing IS EMPTY:
40         db.insert("aci_anomaly_alerts", {
41             alert_id          : generate_uuid(),
42             neighborhood_id   : neighborhood_id,
43             metric_name       : metric_name,
44             observed_value    : latest,
45             expected_value    : mu,
46             z_score           : z_score,
47             severity          : severity,
48             detected_at       : now(),
49             acknowledged      : false,
50             resolved_at       : null
51         })
52         notify_admins(neighborhood_id, metric_name, severity,
53             z_score)

```

Listing 3: Trend Detector — Z-Score Anomaly Detection Pseudocode

## 5.4 Component 4: Report Generator

### Component 4 – Report Generator

**Responsibility:** Compile pre-computed metric snapshots, trend indicators, and anomaly alerts into structured analytical reports and persist them as `aci_analytics_reports`.

```

1 FUNCTION generate_daily_report(neighborhood_id):
2     today      = date_today()
3     yesterday = today - INTERVAL("1d")
4
5     // Fetch current day's snapshots
6     snapshots = db.query("""
7         SELECT metric_name, value FROM aci_metric_snapshots
8         WHERE neighborhood_id = :nbhd
9             AND window          = '24h'
10            AND snapshot_at::date = :today
11 """, nbhd=neighborhood_id, today=today)
12
13     // Fetch active anomaly alerts
14     alerts = db.query("""
15         SELECT metric_name, severity, z_score FROM
16             aci_anomaly_alerts
17         WHERE neighborhood_id = :nbhd
18             AND detected_at::date = :today
19 """, nbhd=neighborhood_id, today=today)
20
21     // Fetch top contributors (users with most posts)
22     top_contributors = db.query("""
23         SELECT author_id, COUNT(*) AS post_count
24         FROM aci_post_events
25         WHERE neighborhood_id = :nbhd
26             AND event_type = 'created'
27             AND occurred_at::date = :today
28         GROUP BY author_id
29         ORDER BY post_count DESC
30         LIMIT 5
31 """, nbhd=neighborhood_id, today=today)
32
33     // Fetch incident category breakdown
34     incident_breakdown = db.query("""
35         SELECT category, COUNT(*) AS count
36         FROM aci_incident_events
37         WHERE neighborhood_id = :nbhd
38             AND event_type = 'submitted'

```

```

38         AND occurred_at::date = :today
39     GROUP BY category
40     """, nbhd=neighborhood_id, today=today)
41
42     payload = {
43         "period"           : today.isoformat(),
44         "kpis"             : dict_from_rows(snapshots),
45         "anomaly_alerts"   : list_from_rows(alerts),
46         "top_contributors" : list_from_rows(top_contributors),
47         "incident_breakdown" : list_from_rows(incident_breakdown),
48     }
49
50     db.insert("aci_analytics_reports", {
51         report_id      : generate_uuid(),
52         neighborhood_id : neighborhood_id,
53         period_start    : yesterday,
54         period_end      : today,
55         report_type     : "daily",
56         payload         : json_encode(payload),
57         generated_at    : now(),
58         generated_by    : "scheduler"
59     })
60
61     RETURN payload

```

Listing 4: Report Generator — Daily Report Generation Pseudocode

## 5.5 Component 5: Analytics API

</>

Component 5 – Analytics API

**Responsibility:** Expose ACI data through RESTful HTTP endpoints with RBAC enforcement.

### Endpoint Specifications:

| Endpoint                                  | Method | Role Required | Description                                           |
|-------------------------------------------|--------|---------------|-------------------------------------------------------|
| <a href="#">/api/v1/analytics/metrics</a> | GET    | moderator+    | Fetch metric snapshots for a neighborhood and window. |

| Endpoint                                                  | Method | Role Required | Description                                     |
|-----------------------------------------------------------|--------|---------------|-------------------------------------------------|
| <a href="#">/api/v1/analytics/reports</a>                 | GET    | moderator+    | List available reports for a neighborhood.      |
| <a href="#">/api/v1/analytics/reports/{id}</a>            | GET    | moderator+    | Retrieve a specific report by ID.               |
| <a href="#">/api/v1/analytics/reports/generate</a>        | POST   | admin         | Trigger an on-demand report for a neighborhood. |
| <a href="#">/api/v1/analytics/alerts</a>                  | GET    | moderator+    | List open anomaly alerts.                       |
| <a href="#">/api/v1/analytics/alerts/{id}/acknowledge</a> | PATCH  | moderator+    | Acknowledge an anomaly alert.                   |
| <a href="#">/api/v1/analytics/alerts/{id}/resolve</a>     | PATCH  | admin         | Mark an anomaly alert as resolved.              |
| <a href="#">/api/v1/analytics/dashboard/summary</a>       | GET    | moderator+    | Aggregated dashboard summary (cached).          |

```

1 FUNCTION GET /api/v1/analytics/dashboard/summary(request):
2     token          = extract_jwt(request.headers.Authorization)
3     user           = verify_jwt(token)           // raises 401 if
                    invalid
4     require_role(user, ["moderator", "admin"]) // raises 403 if
                    insufficient
5
6     neighborhood_id = user.neighborhood_id
7     cache_key       = "metrics:" + neighborhood_id + ":
                    dashboard_summary"
8
9     cached = redis.get(cache_key)
10    IF cached IS NOT NULL:
11        RETURN HTTP_200(json_decode(cached))
12
13    // Build live summary
14    summary = {
15        "dau"                : get_latest_metric(neighborhood_id, "dau
                    ", "24h"),

```

```
16         "mau" : get_latest_metric(neighborhood_id, "mau",  
17         "30d"),  
18         "post_count_24h" : get_latest_metric(neighborhood_id, "  
19         post_count","24h"),  
20         "incident_count" : get_latest_metric(neighborhood_id, "  
21         incident_count","24h"),  
22         "engagement_rate" : get_latest_metric(neighborhood_id, "  
23         engagement_rate","24h"),  
24         "open_alerts" : count_open_alerts(neighborhood_id),  
25         "generated_at" : now().isoformat()  
    }  
  
    redis.set(cache_key, json_encode(summary), ttl_seconds=60)  
    RETURN HTTP_200(summary)
```

Listing 5: Analytics API — Dashboard Summary Endpoint Pseudocode

## 6 Human Interface Design

### 6.1 Overview of User Interface

The ACI module exposes a web-based analytics dashboard accessible from within the Verified neighborhood community platform's admin portal. The dashboard follows a responsive, card-based layout optimized for desktop browsers (minimum 1024px width) and degrades gracefully on tablet-sized screens.

#### Design Principles:

- **Role-filtered views:** Moderators see neighborhood-scoped data only. Platform administrators see cross-neighborhood aggregates.
- **Progressive disclosure:** Summary cards on the main dashboard link to detailed drill-down views.
- **Real-time indicators:** KPI cards refresh every 60 seconds without full page reload (polling the cached dashboard summary endpoint).
- **Alert prominence:** Active anomaly alerts appear in a persistent banner at the top of the dashboard until acknowledged.

### 6.2 Screen Images

## Screen 1 — Main Analytics Dashboard

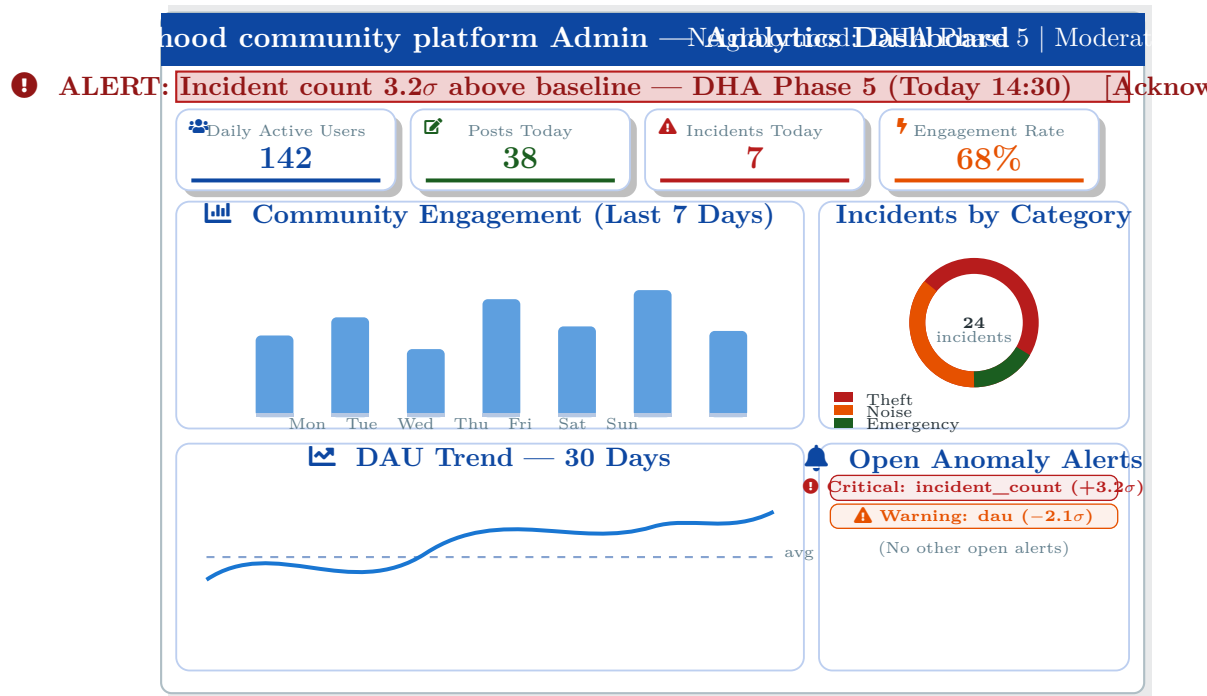


Figure 4: Analytics Dashboard — Main Screen Layout (Wireframe)

## Screen 2 — Report Detail View

### Report Detail View

The Report Detail View displays a full `AnalyticsReport` payload, including KPI table, top-contributor rankings, incident breakdown table, and trend commentary. Accessible via `Reports → [Report Date]` navigation. Includes an **Export PDF** button that calls the backend PDF generation endpoint.

*Navigation path:* Admin Portal → Analytics → Reports → [Report Date]

## Screen 3 — Anomaly Alert Management

### 🔔 Anomaly Alert Management View

A tabular view listing all anomaly alerts for the neighborhood, filterable by severity, metric, and date range. Each row includes: metric name, observed value, expected value, Z-score, severity badge, detection timestamp, and action buttons (**Acknowledge** / **Resolve**).

*Navigation path:* Admin Portal → Analytics → Alerts

### 6.3 Screen Objects and Actions

| UI Element                    | Type                | Action / Behavior                                                                                                                                                                                                                     |
|-------------------------------|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>KPI Card (DAU)</b>         | Display card        | Auto-refreshes every 60 seconds via polling <code>/api/v1/analytics/dashboard/summary</code> . Clicking opens a 30-day trend drill-down modal.                                                                                        |
| <b>Engagement Bar Chart</b>   | Chart.js bar chart  | Displays 7-day post/interaction counts. Hovering a bar shows exact counts in a tooltip.                                                                                                                                               |
| <b>Incident Pie Chart</b>     | Chart.js doughnut   | Displays incident category distribution for the selected period. Clicking a segment filters the incident table below.                                                                                                                 |
| <b>DAU Trend Line</b>         | Chart.js line chart | 30-day rolling DAU with EMA overlay. Toggle buttons switch between 7d / 30d view.                                                                                                                                                     |
| <b>Alert Banner</b>           | Fixed-top div       | Red background; shows the most recent critical alert. Clicking “Acknowledge” calls <code>PATCH /api/v1/analytics/alerts/{id}/ack</code> and hides the banner.                                                                         |
| <b>Alert Table Row</b>        | Table row           | Each row includes a Resolve button (admin only) that calls <code>PATCH /api/v1/analytics/alerts/{id}/resolve</code> and refreshes the table.                                                                                          |
| <b>Generate Report Button</b> | Primary button      | Present on the Reports page (admin only). Calls <code>POST /api/v1/analytics/reports/generate</code> with the selected neighborhood and date range. Shows a spinner until the report is ready, then auto-navigates to the new report. |
| <b>Date Range Picker</b>      | Input group         | Filters all dashboard charts and tables to the selected date range. Triggers re-fetch of metric API endpoints with updated query parameters.                                                                                          |
| <b>Export PDF</b>             | Secondary button    | On the Report Detail page. Triggers a backend PDF render and prompts the browser to download the generated file.                                                                                                                      |

| UI Element            | Type                  | Action / Behavior                                                                                                                                |
|-----------------------|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Neighborhood Selector | Dropdown (admin only) | Visible only to platform administrators. Allows switching the dashboard context to any neighborhood. Moderators see only their own neighborhood. |

## 7 Requirements Matrix

The following table traces each functional requirement of the ACI module (as defined in the companion SP-SRS document) to the design component(s) responsible for its implementation.

| Req. ID      | Functional Requirement                                                                                                                                    | Design Component(s) | Section |
|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|---------|
| <b>FR-01</b> | The system shall collect and ingest domain events from all Verified neighborhood community platform operational modules in real time.                     | Event Consumer      | §5.1    |
| <b>FR-02</b> | The system shall validate event schema before persisting to the analytics store and route invalid events to a dead-letter queue.                          | Event Consumer      | §5.1    |
| <b>FR-03</b> | The system shall compute Daily Active User (DAU) counts per neighborhood on a 1-hour schedule.                                                            | Data Aggregator     | §5.2    |
| <b>FR-04</b> | The system shall compute Monthly Active User (MAU) counts per neighborhood on a 24-hour schedule.                                                         | Data Aggregator     | §5.2    |
| <b>FR-05</b> | The system shall compute post count, engagement rate (likes + comments + shares per post), and incident count per neighborhood per time window.           | Data Aggregator     | §5.2    |
| <b>FR-06</b> | The system shall detect anomalous metric values using Z-score analysis with a 30-day rolling baseline and generate alerts for deviations $\geq 2\sigma$ . | Trend Detector      | §5.3    |

*Continued...*

| Req. ID | Functional Requirement                                                                                                                                          | Design Component(s)             | Section    |
|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------|------------|
| FR-07   | The system shall classify anomaly alerts as <i>warning</i> ( $ z  \geq 2$ ) or <i>critical</i> ( $ z  \geq 3$ ).                                                | Trend Detector                  | §5.3       |
| FR-08   | The system shall prevent duplicate alerts for the same metric within a 2-hour window.                                                                           | Trend Detector                  | §5.3       |
| FR-09   | The system shall generate daily, weekly, and monthly analytical reports per neighborhood including KPIs, top contributors, and incident category distributions. | Report Generator                | §5.4       |
| FR-10   | Authorized administrators shall be able to trigger on-demand report generation for any neighborhood and date range.                                             | Analytics API, Report Generator | §5.4, §5.5 |
| FR-11   | The system shall expose RESTful API endpoints for metric queries, report retrieval, and anomaly alert management.                                               | Analytics API                   | §5.5       |
| FR-12   | Dashboard API endpoints shall enforce RBAC: moderators access their own neighborhood data; platform admins access all neighborhoods.                            | Analytics API                   | §5.5       |
| FR-13   | Dashboard KPI summary responses shall be served within 200ms under normal load (achieved via Redis caching with 60-second TTL).                                 | Analytics API, Redis Cache      | §5.5, §3.3 |
| FR-14   | The system shall display a real-time analytics dashboard with KPI cards, engagement trend chart, incident category chart, and anomaly alert panel.              | Admin Dashboard UI              | §6         |

Continued...

| Req. ID | Functional Requirement                                                                                                                                             | Design Component(s)                  | Section    |
|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------|------------|
| FR-15   | Anomaly alerts shall be displayed prominently (banner) and must be explicitly acknowledged by an authorized user.                                                  | Admin Dashboard UI, Analytics API    | §6.3, §5.5 |
| FR-16   | The system shall allow moderators and admins to resolve anomaly alerts after remediation.                                                                          | Analytics API                        | §5.5       |
| FR-17   | Dashboard charts shall support filtering by date range and shall refresh automatically every 60 seconds.                                                           | Admin Dashboard UI                   | §6.3       |
| FR-18   | Reports shall be exportable as PDF documents.                                                                                                                      | Report Generator, Admin Dashboard UI | §5.4, §6.3 |
| FR-19   | The system shall maintain a time-series data store with partitioned tables to support efficient historical queries.                                                | Analytics DB (PostgreSQL)            | §4.1, §3.3 |
| FR-20   | The system shall supply pre-computed community behavior signals (engagement rate, incident frequency) via API to the AI Moderation and Smart Feed Ranking modules. | Analytics API                        | §5.5       |

## 8 Appendices

### Appendix A — Kafka Topic Schemas

All Kafka messages are serialized as JSON with UTF-8 encoding. Each message must include the following envelope fields:

```

1 {
2     "event_id"      : "<UUID>",          // unique per event
3     "event_type"    : "<string>",        // e.g., "post.created"
4     "topic"         : "<string>",        // Kafka topic name
5     "produced_at"   : "<ISO-8601>",      // event production
6         timestamp
7     "source_service" : "<string>",        // producing service name
8     "payload"       : { ... }           // event-specific fields
9 }
```

Listing 6: Kafka Event Envelope Schema

#### Example — `post.created` event:

```

1 {
2     "event_id"      : "a1b2c3d4-...",
3     "event_type"    : "post.created",
4     "topic"         : " Verified neighborhood community platform.
5         posts.events",
6     "produced_at"   : "2026-03-15T10:22:00Z",
7     "source_service" : "post-service",
8     "payload": {
9         "post_id"      : "f5e6d7c8-...",
10        "author_id"     : "9a8b7c6d-...",
11        "neighborhood_id" : "1e2f3a4b-...",
12        "category"      : "announcement",
13        "has_media"     : true
14    }
15 }
```

### Appendix B — API Error Codes

| Error Code | HTTP Status | Meaning                                                             |
|------------|-------------|---------------------------------------------------------------------|
| ACI-001    | 400         | Invalid query parameter (e.g., unrecognized metric name or window). |
| ACI-002    | 401         | Missing or invalid JWT token.                                       |
| ACI-003    | 403         | Insufficient role for the requested resource.                       |
| ACI-004    | 404         | Report or alert not found for the given ID.                         |
| ACI-005    | 409         | Report already exists for the given neighborhood and period.        |
| ACI-006    | 429         | Rate limit exceeded (100 requests/minute per user).                 |
| ACI-007    | 500         | Internal server error — analytics computation failure.              |
| ACI-008    | 503         | Analytics service temporarily unavailable (e.g., DB unreachable).   |

## Appendix C — Non-Functional Requirements Compliance

| NFR                 | Target                          | Design Mechanism                                                              |
|---------------------|---------------------------------|-------------------------------------------------------------------------------|
| Dashboard load time | < 200 ms (cached)               | Redis cache with 60-second TTL on summary endpoint.                           |
| Availability        | 99% uptime                      | Kafka consumer auto-restart; stateless API containers; DB connection pooling. |
| Data freshness      | Real-time events<br>< 2 min lag | Kafka consumer poll interval 500ms;<br>1-minute aggregation job.              |

| NFR                    | Target                        | Design Mechanism                                                                                |
|------------------------|-------------------------------|-------------------------------------------------------------------------------------------------|
| <b>Security</b>        | RBAC + encrypted transit      | JWT validation at API layer; TLS on all connections; no PII in analytics schema beyond user_id. |
| <b>Scalability</b>     | Horizontal scale for API tier | Stateless API containers deployable as multiple instances behind load balancer.                 |
| <b>Maintainability</b> | Modular component design      | Each of the 5 ACI components is independently deployable and testable.                          |
| <b>Audit logging</b>   | All admin actions logged      | Every API write action logged with user_id, action, timestamp, and outcome.                     |

## Appendix D — Revision History

| Version | Date        | Author      | Description                                                        |
|---------|-------------|-------------|--------------------------------------------------------------------|
| 0.1     | 12 Mar 2026 | Hamad Khan  | Initial draft outline.                                             |
| 0.5     | 20 Mar 2026 | Shah Faisal | Architecture and data design sections completed.                   |
| 0.8     | 25 Mar 2026 | Ali Sajjad  | Component pseudocode and API table added.                          |
| 0.9     | 26 Mar 2026 | Shace Tariq | UI design and dashboard wireframe added.                           |
| 1.0     | 27 Mar 2026 | Shah Faisal | Final review, requirements matrix completed, ready for submission. |